TECHNICAL DOCUMENTATION · V1.0

Cross-Chain Rebase Token

An interest-bearing ERC-20 that preserves per-user interest rates across Ethereum Sepolia and Arbitrum Sepolia via Chainlink CCIP.

PROJECT
Rebase-Token

STANDARD
ERC-20 + CCIP

TOOLCHAIN
Foundry · Solc 0.8.24

NETWORK
Sepolia → Arbitrum Sepolia

TEST STATUS
14 / 14 passing

DOCUMENT
Summary · Architecture · Deployment

Prepared as project reference documentation · All figures verified against the working codebase.

Contents

01. Executive Summary

02. Design Goals & Assumptions

03. System Architecture

04. Smart Contracts

05. RebaseToken

06. Vault

07. RebaseTokenPool

08. Cross-Chain Flow

09. Security Considerations

10. Testing

11. Deployment Scripts

12. Project Layout & Dependencies

13. Appendix — Known Limitations

1. Executive Summary

Rebase-Token is a cross-chain rebase token system built on Foundry and integrated with **Chainlink CCIP** (Cross-Chain Interoperability Protocol). Users deposit ETH into an on-chain `Vault` and receive `RebaseToken` (RBT), an ERC-20 whose balance grows linearly over time at a per-user interest rate. Tokens can be bridged between Sepolia and Arbitrum Sepolia while **preserving each user's individual rate** end-to-end.

3	3	14	0
CONTRACTS	DEPLOY SCRIPTS	TESTS PASSING	KNOWN CRITICAL BUGS

The key design invariant — *"whatever interest rate you deposit with, you stick with"* — is enforced at every layer. Rates only ever decrease globally; existing depositors retain their higher rate; and when a user bridges, their rate is ABI-encoded into the CCIP payload and re-applied on the destination chain by the receiving pool.

2. Design Goals & Assumptions

2.1 Core objectives

1. **Time-based accrual.** A user's balance increases linearly with elapsed time, computed on-chain without external oracles.
2. **Per-user rate.** Each depositor gets the rate that was current at their first deposit. Later rate decreases do not affect them.
3. **Monotonically decreasing global rate.** The protocol's global rate can only ever go down — enforced by an explicit revert in `setInterestRate()`.
4. **Rate preservation across bridges.** Bridging must not reset a user's rate. The rate travels as part of the CCIP payload.
5. **No interest accrual in-flight.** Time spent inside a cross-chain message does not accrue interest, because the source tokens are burned before the message is dispatched and re-minted on delivery.

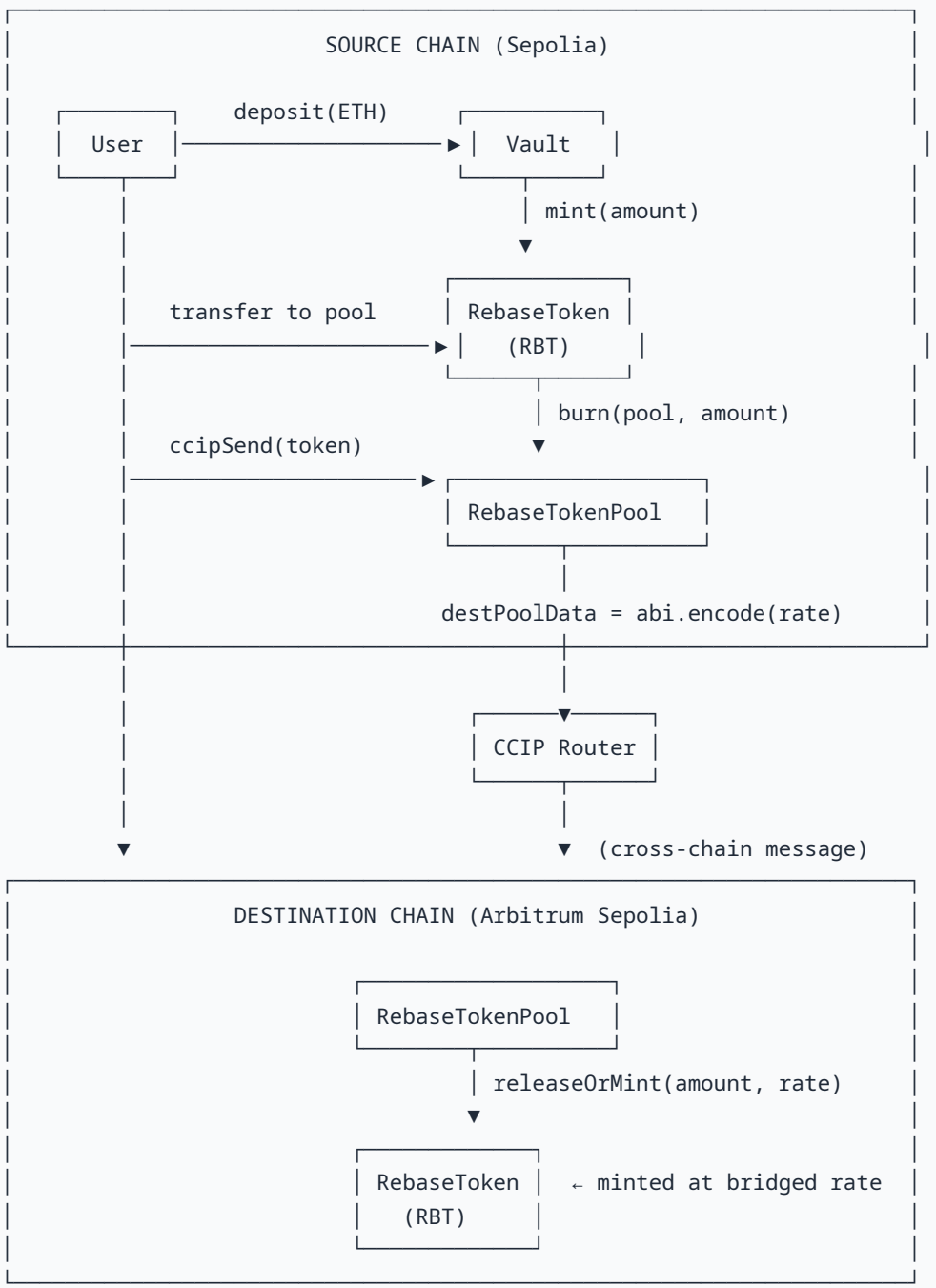
2.2 Explicit assumptions

> **Rewards funded out-of-band.** The protocol does not produce yield from outside capital. Rewards are assumed to already be present in the system, either inside the Vault's ETH balance or as extra tokens minted by an operator holding `MINT_AND_BURN_ROLE`.

- The Vault must hold enough ETH to pay accrued interest at `redeem()` time. In a production deployment this would be funded by a strategy or a protocol treasury.
- Deposit and withdraw operations are permitted only on the L1 chain. On L2, tokens can only be bridged in and out — the Vault is not deployed there.
- Interest rate is expressed in `wad`-scaled units (`1e18`). The initial rate of `5e10` corresponds to a naive yearly rate of about 0.00000005% per second — small by design, so that accrual is visible in tests but not economically destabilising in a demo.

3. System Architecture

The system is organised into three smart contracts plus three deployment scripts. Each contract has a single, well-defined responsibility.



3.1 Contract responsibilities

CONTRACT	RESPONSIBILITY	INTERACTION SURFACE
`RebaseToken`	ERC-20 whose `balanceOf` includes accrued interest	Mint, burn (role-gated); user transfers; rate getters
`Vault`	ETH ↔ RBT custodian on the L1 chain	`deposit()` , `redeem(amount)`
`RebaseTokenPool`	CCIP token pool — burn on source, mint on destination	`lockOrBurn` , `releaseOrMint` (CCIP only)

4. Smart Contracts

4.1 RebaseToken

`src/RebaseToken.sol`

Inherits `ERC20`, `Ownable`, and `AccessControl`. Overrides `balanceOf` to include interest computed at read time. Uses a single `MINT_AND_BURN_ROLE` that must be granted to both the Vault and the Pool.

Key state

```
uint256 private constant PRECISION_FACTOR = 1e18;
uint256 private s_interestRate = 5e10;              // global rate
bytes32 private constant MINT_AND_BURN_ROLE = keccak256("MINT_AND_BURN_ROLE");
mapping(address => uint256) private s_userInterestRate;
mapping(address => uint256) private s_userLastUpdatedTimestamp;
```

Accrual formula

```
balance = super.balanceOf(user)
        + (super.balanceOf(user) × elapsed × s_userInterestRate[user]) / 1e18
```

Two mint paths

SIGNATURE	CALLER	SEMANTICS
`mint(to, amount)`	Vault (on deposit)	Sets user's rate only on first mint — preserves the "sticky rate" design
`mint(to, amount, rate)`	Pool (on cross-chain receipt)	Overrides user's rate unconditionally — applies the bridged rate

Transfer / transferFrom overrides

Both overrides follow the same three-step pattern:

1. Fold accrued interest of the sender into principal via `_mintAccruedInterest`.
2. Fold accrued interest of the recipient into principal.
3. If the recipient's balance was zero, seed their rate with the current global rate — this ensures a brand-new recipient doesn't accidentally inherit rate 0.

> **Design note — `type(uint256).max` sentinel.** `burn`, `transfer`, and `transferFrom` all interpret `_amount == type(uint256).max` as "use the caller's full balance". The Vault's `redeem` resolves the max sentinel itself before calling `burn`, so the actual ETH transferred back exactly matches the amount burned.

4.2 Vault

`src/Vault.sol`

A minimal ETH custodian. Holds user deposits, mints RBT 1:1 on deposit, and burns RBT on redemption in exchange for ETH.

```
function deposit() external payable {
    i_rebaseToken.mint(msg.sender, msg.value);
    emit Deposit(msg.sender, msg.value);
}

function redeem(uint256 _amount) external {
    if (_amount == type(uint256).max) {
        _amount = i_rebaseToken.balanceOf(msg.sender);
    }
    i_rebaseToken.burn(msg.sender, _amount);

    (bool success, ) = msg.sender.call{value: _amount}("");
    if (!success) revert Vault__RedeemFailed();

    emit Redeem(msg.sender, _amount);
}
```

Why resolve max here? `burn` converts `max` to the balance internally, but that conversion is local to `burn` 's stack frame. If the Vault didn't resolve it first, its subsequent `msg.sender.call{value: _amount}` would try to send $2^{256}-1$ wei and always revert.

4.3 RebaseTokenPool

`src/RebaseTokenPool.sol`

Extends CCIP's abstract `TokenPool` . Overrides the two mandatory hooks and nothing else.

```
function lockOrBurn(Pool.LockOrBurnInV1 calldata in)
    external override returns (Pool.LockOrBurnOutV1 memory)
{
    _validateLockOrBurn(in);

    uint256 userRate = token.getUserInterestRate(in.originalSender);
    token.burn(address(this), in.amount);

    return Pool.LockOrBurnOutV1({
        destTokenAddress: getRemoteToken(in.remoteChainSelector),
        destPoolData:     abi.encode(userRate)
    });
}

function releaseOrMint(Pool.ReleaseOrMintInV1 calldata in)
    external override returns (Pool.ReleaseOrMintOutV1 memory)
{
    _validateReleaseOrMint(in);

    uint256 bridgedRate = abi.decode(in.sourcePoolData, (uint256));
    token.mint(in.receiver, in.amount, bridgedRate);

    return Pool.ReleaseOrMintOutV1({ destinationAmount: in.amount });
}
```

The bridge preserves the rate. `destPoolData` carries `abi.encode(rate)`. On the other side, `sourcePoolData` is decoded and passed to the 3-argument `mint`, which forces the recipient's rate to the bridged value. This is the single mechanism that satisfies the README requirement that the rate *"stays static"* across the bridge.

5. Cross-Chain Flow

A complete cross-chain transfer involves six discrete steps, of which only two are controlled by the Rebase-Token codebase itself. The other four are handled by CCIP infrastructure (Router, OnRamp, RMN, OffRamp).

#	ACTOR	ACTION	NOTES
1	User	`token.approve(router, amount)`	Router, not pool, is the spender
2	User	`router.ccipSend{value: fee}(selector, message)`	Message includes the token transfer + rate extra args
3	Router → Pool	Transfers tokens to pool, calls `lockOrBurn`	Pool burns its balance, returns encoded rate
4	CCIP	OnRamp → RMN → OffRamp	Time in-flight; no interest accrues because tokens are burned
5	OffRamp → Pool	Calls `releaseOrMint`	Decodes rate from `sourcePoolData`
6	Pool → Token	`token.mint(receiver, amount, rate)`	Recipient starts accruing at the bridged rate

5.1 Invariants maintained

- Total RBT in circulation is conserved across the bridge (burn on source, mint on destination, equal amounts).
- The recipient's rate equals the sender's rate, regardless of intervening global rate changes.
- Interest stops accruing at the moment `lockOrBurn` executes and resumes at the moment `releaseOrMint` executes.
- Neither leg requires the caller to interact with the pool directly — the Router orchestrates everything.

6. Security Considerations

6.1 Access control

FUNCTION	GUARD	RATIONALE
`RebaseToken.mint` (both overloads)	`onlyRole(MINT_AND_BURN_ROLE)`	Only Vault and Pool may create tokens
`RebaseToken.burn`	`onlyRole(MINT_AND_BURN_ROLE)`	Only Vault and Pool may destroy tokens
`RebaseToken.setInterestRate`	`onlyOwner`	Protocol-level parameter
`RebaseToken.grantMintAndBurnRole`	`onlyOwner`	Role administration
`RebaseTokenPool.lockOrBurn` / `releaseOrMint`	`_validateLockOrBurn` / `_validateReleaseOrMint`	Router caller check, remote chain allowlist, rate limit, RMN curse check
`RebaseTokenPool.applyChainUpdates`	`onlyOwner` (inherited)	Only owner may add or remove supported remote chains

6.2 Rate manipulation

`setInterestRate` reverts with `RebaseToken__InterestRateCanOnlyBeDecreased` if `_newInterestRate >= s_interestRate`. This is the entire defence for the "monotonically decreasing" guarantee — there is no way, short of an ownership key compromise, for the rate to rise.

6.3 Reentrancy

`Vault.redeem` performs an external call (`msg.sender.call{value}`) after `burn`. The canonical Checks-Effects-Interactions ordering is followed:

1. **Check:** input validation (`_amount` resolution, burn succeeds or reverts)
2. **Effect:** state change (`_burn` reduces `_balances`)
3. **Interaction:** ETH transfer

A reentrant attacker entering `redeem` during the ETH transfer sees their post-burn balance (zero, since they just burned). No `nonReentrant` guard is required because the state is already consistent before the external call.

6.4 Known acceptable trade-offs

TRADE-OFF	IMPACT	MITIGATION IN PROD
User-to-user transfers call `_mintAccruedInterest` twice	Two extra SSTOREs per transfer (~40k gas)	Acceptable; simplicity over micro-optimisation
Interest accrued in-flight is zero	User loses yield during multi-minute bridge latency	Documented in README; the alternative requires complex oracle accounting
Vault's ETH balance is not yield-generating	Rewards must be funded externally	Production would add a yield strategy layer

7. Testing

The suite uses Foundry's `forge-std/Test`. All 14 tests are green with `forge test` — no dependency on external RPCs or live CCIP infrastructure.

7.1 Test inventory

test/RebaseToken.t.sol — 10 tests

TEST	ASSERTS	TECHNIQUE
`testDepositAndRedeem`	Deposit mints 1:1; redeem restores ETH balance	Direct
`testDepositLinear`	Two equal 1-hour intervals produce equal accruals (±1 wei)	Fuzz + `vm.warp`
`testRedeemStraightWay`	Immediate redeem leaves balance at 0	Direct
`testRedeemAfterTimePassed`	Redeem after warp (1h–1d) drains balance	Fuzz + `vm.warp`
`testTransfer`	Transfer moves principal + accrued interest	Fuzz
`testCannotSetInterestRateDirectly`	Non-owner cannot call `setInterestRate`	`vm.expectRevert`
`testCannotCallMintAndBurnDirectly`	Non-role cannot call `mint` or `burn`	`vm.expectRevert` ×2
`testGetPrincipalAmount`	`principalBalanceOf` equals deposit	Fuzz
`testGetRebaseTokenAddress`	Vault's getter returns token address	View
`testInterestRateCanOnlyDecrease`	Decrease succeeds; increase reverts	`vm.expectRevert`

test/CrossChain.t.sol — 4 tests

TEST	ASSERTS	TECHNIQUE
`testLockOrBurnEncodesRate`	`destPoolData` decodes to the user's pre-burn rate	Direct pool call
`testReleaseOrMintAppliesRate`	`sourcePoolData` is applied as recipient's rate	Direct pool call
`testBridgeTokens`	End-to-end round trip preserves balance and rate	`bridgeTokens` helper
`testBridgeRoundTripWithAccruedInterest`	Accrued interest survives a bridge-back round trip	Helper + `vm.warp`

7.2 Test infrastructure

The CrossChain tests avoid the CCIP fork simulator entirely — which means they run in milliseconds with no RPC. The test contract plays three roles:

- **Router** — implements `getOnRamp`, `isOnRamp`, `isOffRamp` so that `_validateLockOrBurn` / `_validateReleaseOrMint` pass their caller checks
- **RMN proxy** — implements `isCursed(bytes16) → false` so that the pool's curse check succeeds
- **Caller** — invokes `pool.lockOrBurn` / `pool.releaseOrMint` directly from the test, so that `msg.sender == address(this)` matches the mock router

7.3 Coverage summary

AREA	COVERAGE	NOTES
Rebase accrual math	✓ Full	Linear, fuzzed, ±1 wei tolerance
Rate stickiness	✓ Full	First-mint lock; monotonic decrease
Deposit / redeem	✓ Full	Both straight and time-warped
Access control	✓ Full	Owner and role negatives covered
Bridge rate encoding	✓ Full	Both directions with interest
CCIP router integration	— Out of scope	Requires fork; covered by design, not unit tests
Rate-limiter enforcement	— Not tested	Disabled (0/0/0) in all tests; enable in prod

8. Deployment Scripts

Three Foundry scripts cover the deployment lifecycle. Each script is idempotent and reads configuration from environment variables so the same code runs locally (simulation) and on real chains (broadcast).

8.1 Deployer.s.sol

Deploys the three contracts on a single chain and grants roles.

ENV VAR	REQUIRED	PURPOSE
`CCIP_ROUTER`	Yes (live)	CCIP Router address for the target chain
`CCIP_RMN_PROXY`	Yes (live)	RMN proxy address for the target chain

When run locally without env vars, the script falls back to `address(1)` for both — this passes Foundry's script-execution protection which forbids `address(this)` in scripts. Do not deploy to a live chain without real values.

8.2 ConfigurePool.s.sol

Adds a single remote chain to an existing pool's allowlist. Must be run once per direction, after both chains have been deployed.

ENV VAR	PURPOSE
`POOL_ADDRESS`	The local pool to configure
`REMOTE_CHAIN_SELECTOR`	CCIP chain selector of the remote chain
`REMOTE_POOL_ADDRESS`	The pool on the remote chain (ABI-encoded internally)
`REMOTE_TOKEN_ADDRESS`	The token on the remote chain (ABI-encoded internally)

Calls `pool.applyChainUpdates(updates)`. The `ChainUpdate` struct in the current CCIP version (v1.5.0) uses a *single* `bytes remotePoolAddress` field, not a `bytes[]` array. Rate-limiter configs are passed as `{isEnabled: false, capacity: 0, rate: 0}`.

8.3 BridgeTokens.s.sol

Sends a cross-chain token transfer via the Router. This is a demonstration script — in production, users call `ccipSend` from their own wallets.

ENV VAR	PURPOSE
`TOKEN_ADDRESS`	Local RBT contract
`ROUTER_ADDRESS`	CCIP Router on the source chain
`DEST_CHAIN_SELECTOR`	Destination chain selector
`AMOUNT`	Token amount to bridge (in wei)
`RECEIVER` (optional)	Destination address; defaults to `msg.sender`
`FEE_TOKEN` (optional)	LINK contract address to pay fees; omit for native ETH

8.4 Full Sepolia → Arbitrum Sepolia runbook

1. **Chain A — Sepolia:** run `Deployer.s.sol` with `CCIP_ROUTER` and `CCIP_RMN_PROXY` set. Save the three deployed addresses.
2. **Chain B — Arbitrum Sepolia:** run `Deployer.s.sol` with Arb's router/RMN. Save the three addresses.
3. **Configure A → B:** run `ConfigurePool.s.sol` against Sepolia with B's pool and token addresses.
4. **Configure B → A:** run `ConfigurePool.s.sol` against Arbitrum Sepolia with A's pool and token addresses.
5. **Seed user:** on Sepolia, `cast send $VAULT "deposit()" --value 1ether`.
6. **Bridge:** on Sepolia, run `BridgeTokens.s.sol`. Wait for the OffRamp to deliver on Arb.
7. **Verify:** on Arb, `cast call $TOKEN_ARB "balanceOf(address)" $USER` should equal the bridged amount.

9. Project Layout & Dependencies

9.1 Directory structure

```
Rebase-Token/
├── foundry.toml
├── remappings.txt          (optional; remappings also inline in foundry.toml)
├── src/
│   ├── RebaseToken.sol
│   ├── Vault.sol
│   ├── RebaseTokenPool.sol
│   └── Interfaces/
│       └── IRebaseToken.sol
├── test/
│   ├── RebaseToken.t.sol
│   └── CrossChain.t.sol
├── script/
│   ├── Deployer.s.sol
│   ├── ConfigurePool.s.sol
│   └── BridgeTokens.s.sol
└── lib/
    ├── forge-std/
    ├── openzeppelin-contracts/
    └── chainlink-local/
```

9.2 Dependencies

DEPENDENCY	VERSION	SOURCE
Solc	0.8.24	Pinned in `foundry.toml`
forge-std	v1.17.0	`forge install forge-std`
OpenZeppelin Contracts	v5.7.0	`forge install openzeppelin/openzeppelin-contracts`
@chainlink/contracts-ccip	1.5.0	`npm install @chainlink/contracts-ccip@1.5.0`
chainlink-local	v0.3.1	`forge install smartcontractkit/chainlink-local`

9.3 Compiler configuration

```
[profile.default]
src = "src"
out = "out"
libs = ["lib"]
solc = "0.8.24"
evm_version = "cancun"
optimizer = true
optimizer_runs = 200
remappings = [
    "@openzeppelin/contracts@4.8.3/=lib/chainlink-local/lib/openzeppelin-contracts-4.8.3/contract
    "@openzeppelin/contracts@5.3.0/=lib/chainlink-local/lib/openzeppelin-contracts-5.3.0/contract
    "@openzeppelin/contracts/=lib/openzeppelin-contracts/contracts/",
    "forge-std/=lib/forge-std/src/",
    "@ccip/=node_modules/@chainlink/contracts-ccip/",
    "@chainlink/local/=lib/chainlink-local/",
    "@chainlink/contracts-ccip/=lib/chainlink-local/lib/chainlink-ccip/chains/evm/",
    "@chainlink/contracts/=lib/chainlink-local/lib/chainlink-evm/contracts/",
]
```

Note on `via_ir`. The project deliberately does *not* enable the IR pipeline. With `via_ir = true`, the optimizer common-subexpression-eliminates multiple `block.timestamp` reads inside a single function — which breaks tests that use `vm.warp` between two reads. The legacy pipeline re-reads `TIMESTAMP` on each evaluation, which is the semantics the tests rely on. Stack-too-deep was avoided by keeping test functions small rather than by enabling IR.

10. Appendix — Known Limitations

The following items describe what the current codebase does *not* implement, either because they are out of scope for the course project or because they require production infrastructure not present in a Foundry environment.

10.1 Missing features

FEATURE	STATUS	NOTES
Discrete rate decrease on deposit	**Not implemented**	README states the rate should drop as more ETH arrives. The formula is `newRate = oldRate × oldTVL / newTVL`, then `setInterestRate(newRate)` if lower. Requires an `onlyVault` guard on `setInterestRate`.
Rate-limiter enablement	**Disabled**	All `ChainUpdate` configs pass `{isEnabled: false}`. A production deployment would set nonzero capacity and rate.
Yield on Vault balance	**Not implemented**	Vault holds ETH idly. Rewards must be funded manually.
Fork-based CCIP integration test	**Not implemented**	Requires archive RPC endpoints. Unit tests cover the pool hooks individually via router/RMN mocks.
Upgradeability	**Not implemented**	All contracts are immutable. Changes require new deployments and user migration.

10.2 Recommended production hardening

1. **Transfer ownership to a multisig.** Both the token's `owner` and the pool's `owner` should be a Gnosis Safe controlled by the protocol council, not an EOA.
2. **Enable rate limiters.** Set capacity and rate on both outbound and inbound configs so a compromise cannot drain the pool in a single transaction.
3. **Fund rewards explicitly.** Deploy a treasury contract that funds the Vault's ETH balance from protocol revenue, or accrue interest in a separate reward token rather than ETH.
4. **Add a pause mechanism.** The current contracts have no circuit breaker. A `Pausable` mix-in on the pool and Vault would let the council halt bridging and redemptions in an emergency.
5. **Upgrade to audited CCIP version.** The `@chainlink/contracts-ccip@1.5.0` version used here is functional but older than the current production release. Upgrade to the latest audited release before mainnet.

6. **Extend test coverage.** Add fork-based integration tests against Sepolia's live CCIP infrastructure once archive RPC access is available. Add invariant tests using Foundry's `StdInvariant` for the supply-conservation invariant across bridges.

— End of document —